

Task-3: SQL For Data Analysis

Objective: Use SQL queries to extract and analyze data from a database.

Tools: MySQL or PostgreSQL or SQLite

Deliverables: SQL queries in a SQL file + screenshots of output

The screenshot shows a SQL client interface with a dark theme. The main window displays a query and its results. The query is:

```
1 -- 1. View all data
2 SELECT * FROM cleaned_defence_data;
```

The results are displayed in a table with 8 columns: ID, Description, Code, A, Revised, Budget, B, Budget. The table contains 8 rows of data:

ID	Description	Code	A	Revised	Budget	B	Budget
1.0	Pay and Allowances of the Army	2076		49647...	49647...		54446...
2.0	Pay and Allowances and Miscellaneous Expenses of the Auxiliary For...	2076		967.53	967.53		1056...
3.0	Pay and Allowances of Civilians	2076		4041.8	4041.8		4410...
4.0	Transportation	2076		2676...	2676...		2368.0
5.0	Military Farms	2076		494.96	494.96		428.77
6.0	Ex-Servicemen Contributory Health Scheme	2076		1781...	1781...		1420...
7.0	Inspection	2076		760.96	760.96		832.8
8.0	Stores	2076		13955...	13955...		15198...

The interface also includes a sidebar on the left with a list of databases (SQLite, MariaDB, PostgreSQL, MS SQL) and a right sidebar showing the query history.

2.

The screenshot shows a database management interface with a SQLite query editor and a results pane. The query is:

```
1 -- 2. View limited rows (top 5 rows)
2 SELECT * FROM cleaned_defence_data
3 LIMIT 5;
```

The results pane displays a table with 8 columns: ID, Description, Code, A, Revise, Budget, B, Budget. The data is as follows:

ID	Description	Code	A	Revise	Budget	B	Budget
1.0	Pay and Allowances of the Army	2076		49647...	49647...		54446...
2.0	Pay and Allowances and Miscellaneous Expenses of the Auxiliary For...	2076		967.53	967.53		1056...
3.0	Pay and Allowances of Civilians	2076		4041.8	4041.8		4410...
4.0	Transportation	2076		2676...	2676...		2368.8

The History pane on the right shows the executed queries and their timestamps.

3.

The screenshot shows a database management interface with a SQLite query editor and a results pane. The query is:

```
1 -- 3. View specific columns
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c14 AS REAL) AS Budget_2016_2017
6 FROM cleaned_defence_data;
```

The results pane displays a table with 4 columns: Scheme, Budget_2015_2016, Budget_2016_2017. The data is as follows:

Scheme	Budget_2015_2016	Budget_2016_2017
Code	0	0
2076	49647.95	60548.18
2076	967.53	1172.2
2076	4041.8	4878.8
2076	2676.16	2828.1
2076	494.96	354.1
2076	1781.39	2639
2076	760.96	851.04
2076	13955.3	16697.84
2076	6327.95	8259
2076	4436.32	5453
2076	874.47	1016.39

The History pane on the right shows the executed queries and their timestamps.

4.

The screenshot shows a database management tool interface. On the left, there's a sidebar with a list of databases: SQLite (0.1.4 beta), MariaDB, PostgreSQL, and MS SQL. The main area displays a SQLite query:

```
1 -- 4. List unique values (Distinct Descriptions)
2 SELECT DISTINCT c2 AS Unique_Descriptions
3 FROM cleaned_defence_data;
```

The results are shown in a table with one column, 'Unique_Descriptions', containing the following values:

Unique_Descriptions
Description
Pay and Allowances of the Army
Pay and Allowances and Miscellaneous Expenses of the Auxiliary Forces
Pay and Allowances of Civilians
Transportation
Military Farms
Ex-Servicemen Contributory Health Scheme
Inspection
Stores

On the right, a 'History' panel shows a list of executed queries with their timestamps.

5.

The screenshot shows the same database management tool interface. The main area displays a SQLite query:

```
1 -- 5. Use WHERE to filter rows (where revised budget is greater than original)
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c7 AS REAL) AS Revised_2015_2016
6 FROM cleaned_defence_data
7 WHERE CAST(c7 AS REAL) > CAST(c6 AS REAL);
```

The results are shown in a table with three columns: 'Scheme', 'Budget_2015_2016', and 'Revised_2015_2016'. The data is as follows:

Scheme	Budget_2015_2016	Revised_2015_2016
0076	-1964.72	0

On the right, the 'History' panel shows a list of executed queries with their timestamps.

6.

The screenshot shows a database management tool interface. On the left, there's a sidebar with a list of databases: SQLite (selected), MariaDB, PostgreSQL, and MS SQL. The main area displays a SQL query and its results. The query is as follows:

```
1 -- 6. Combine conditions with AND/OR
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c14 AS REAL) AS Budget_2016_2017
6 FROM cleaned_defence_data
7 WHERE CAST(c6 AS REAL) > 5000 AND CAST(c14 AS REAL) > 10000;
```

The results are shown in a table with the following columns: Scheme, Budget_2015_2016, and Budget_2016_2017.

Scheme	Budget_2015_2016	Budget_2016_2017
2076	49647.95	60548.18
2076	13955.3	16697.84
	85785.82	104158.95

On the right, there's a 'History' panel showing previous queries and their execution times.

7.

The screenshot shows the same database management tool interface. The SQL query is now as follows:

```
1 -- 7. Sort results in descending order (by budget increase)
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CAST(c14 AS REAL) AS Budget_2016_2017,
6   (CAST(c14 AS REAL) - CAST(c6 AS REAL)) AS Increase
7 FROM cleaned_defence_data
8 ORDER BY Increase DESC
9 LIMIT 5;
```

The results are shown in a table with the following columns: Scheme, Budget_2015_2016, Budget_2016_2017, and Increase.

Scheme	Budget_2015_2016	Budget_2016_2017	Increase
	85785.82	104158.95	18373.129999999999
2076	49647.95	60548.18	10900.230000000003
2076	13955.3	16697.84	2742.5400000000001
2076	6327.95	8259	1931.0500000000002
2076	4436.32	5453	1016.6800000000003

The 'History' panel on the right shows the previous query and its execution time.

8.

The screenshot shows a database management interface with a dark theme. On the left, a sidebar lists databases: SQLite (selected), MariaDB, PostgreSQL, and MS SQL. Below the sidebar, a table named 'cleaned_defence_data' is shown with a single row containing the value '15' under the column 'Total_Rows'. The main area displays a SQL query: `1 -- 8. Count total number of rows`, `2 SELECT COUNT(*) AS Total_Rows`, `3 FROM cleaned_defence_data;`. The right panel shows a history of queries, including the current one and two previous ones. The bottom status bar indicates the system temperature is 34°C and the date is 24-04-2025.

9.

The screenshot shows the same database management interface. The main area displays a SQL query: `1 -- 9. Aggregate: Sum, Avg, Max, Min of 2016 Budget`, `2 SELECT`, `3 SUM(CAST(c14 AS REAL)) AS Total_2016,`, `4 AVG(CAST(c14 AS REAL)) AS Average_2016,`, `5 MAX(CAST(c14 AS REAL)) AS Max_2016,`, `6 MIN(CAST(c14 AS REAL)) AS Min_2016`, `7 FROM cleaned_defence_data;`. The right panel shows a history of queries, including the current one and two previous ones. The bottom status bar indicates the system temperature is 34°C and the date is 24-04-2025.

10.

SQLite

```

1 -- 10. Group and aggregate by Description
2 SELECT
3   c2 AS Description,
4   COUNT(*) AS Count_Entries,
5   SUM(CAST(c14 AS REAL)) AS Total_2016_Budget
6 FROM cleaned_defence_data
7 GROUP BY c2
8 ORDER BY Total_2016_Budget DESC;

```

Description	Count_Entries	Total_2016_Budget
Grand Total arate document.	1	104150.95
Pay and Allowances of the Army	1	60548.18
Stores	1	16697.84
Works	1	8259
Rashtriya Rifles	1	5453
Pay and Allowances of Civilians	1	4878.8
Transportation	1	2828.1
Ex-Servicemen Contributory Health Scheme	1	2639
Other Expenditure	1	2298.18
Pay and Allowances and Miscellaneous Ex.	1	1172.2
National Cadet Corps	1	1016.96

11.

SQLite

```

1 -- 11. Rename columns in result using AS
2 SELECT
3   c2 AS Description,
4   CAST(c14 AS REAL) AS Budget_2016_2017
5 FROM cleaned_defence_data
6 ORDER BY Budget_2016_2017 DESC
7 LIMIT 5;
8

```

Description	Budget_2016_2017
Grand Total arate document.	104150.95
Pay and Allowances of the Army	60548.18
Stores	16697.84
Works	8259
Rashtriya Rifles	5453

12.

The screenshot shows a database management tool interface. On the left, there's a sidebar with a tree view showing a database named 'cleaned_defence_data' with a table 'demo'. The main area displays a SQL query:


```

1 -- 12. Conditional logic using CASE WHEN
2 SELECT
3   c3 AS Scheme,
4   CAST(c6 AS REAL) AS Budget_2015_2016,
5   CASE
6     WHEN CAST(c6 AS REAL) > 10000 THEN 'High Budget'
7     WHEN CAST(c6 AS REAL) BETWEEN 5000 AND 10000 THEN 'Medium Budget'
8     ELSE 'Low Budget'
9   END AS Budget_Category
10 FROM cleaned_defence_data;
  
```

 Below the query, the results are shown in a table:

Scheme	Budget_2015_2016	Budget_Category
Code	0	Low Budget
2076	49647.95	High Budget
2076	967.53	Low Budget
2076	4841.8	Low Budget
2076	2676.16	Low Budget
2076	494.96	Low Budget
2076	1781.39	Low Budget
2076	768.96	Low Budget
2076	13955.3	High Budget

 On the right, there's a 'History' panel showing previous queries and their execution times. The bottom status bar shows the system clock as 20:03 on 24-04-2025.

13.

The screenshot shows the same database management tool interface. The SQL query is:


```

1 -- 13. Subquery to get schemes with highest 2016 budget compared to average
2 SELECT
3   c3 AS Scheme,
4   CAST(c14 AS REAL) AS Budget_2016_2017
5 FROM cleaned_defence_data
6 WHERE CAST(c14 AS REAL) > (
7   SELECT AVG(CAST(c14 AS REAL))
8   FROM cleaned_defence_data
9 )
10 ORDER BY Budget_2016_2017 DESC;
  
```

 The results table is:

Scheme	Budget_2016_2017
	184158.95
2076	60548.18
2076	16697.84

 The 'History' panel on the right shows previous queries. The bottom status bar shows the system clock as 17:28 on 24-04-2025.